Chapter 8: Introduction and Overview

Concurrency is difficult even in a single machine, and it is even more difficult in a distributed system.

- In a concurrent system, processors use the shared memory to exchange the information.
- In a distributed system, each processor has its local state and the participants have to communicate by exchanging messages.

Concurrent and Parallel

- Concurrent: Happening simultaneously, but only one task is being executed at a time.
- Parallel: Multiple tasks are being executed simultaneously.

Shared State in a Distributed System

Even if we introduce a "shared state"-equivalent in a distributed system, it is not going to be the same as a shared state in a single machine.

To access the shared state in a distributed system, the processes still have to go through the communication medium.

- Aspect 1: How long do we wait for a response from the other process?
  - This is defined by system's "synchony" - whether the system is asynchronous or there are any timing assumptions.
- Aspect 2: We don't know the reason for the delay.
  - It could be because the other process is busy, or it could be because the other process is dead.
  - It could be because the network is slow, or it could be because the network is partitioned.
  - This is defined by system's "failure model" - what kind of failures can occur and how they are handled.
- Aspect 3: System reliability
  - This is defined by system's "fault tolerance" - how the system behaves in the presence of faults.

Fallacies of Distributed Computing

The fallacies of distributed computing are a set of assertions made by L Peter Deutsch and others at Sun Microsystems describing false assumptions that programmers new to distributed applications invariably make. Source: Wikipedia

Unfortunately, there are more assumptions that people make.

Processing

- We cannot assume that processing is instantaneous.
- The message may join the pending queue on the remote server, and will be processed only after some time.

Backpressure:

- Backpressure refers to the resistance or force opposing the desired flow of data through software. Source
- It is a strategy that allows us to cope with producers that publish messages at a faster rate than what the consumers can process, by slowing down the producers.

Clocks and Time

We cannot assume that clocks on different remote machines are in sync.

State Consistency

- We have to be cautious of false assumptions and avoid oversimplification in understanding distributed systems.
- Loose Constraints in Distributed Algorithms: Some algorithms allow state divergence. In such cases, conflict resolution and read-time data repair are crucial.
- Eventually consistent distributed DBMS cannot assume that the schema is consistent across all nodes.

Local and Remote Execution

- Hiding complexity behind APIs for remote datasets can lead to issues.
- Using the same interface for local and remote execution may be misleading.
- Remote invocation is much costlier than local calls.
- Interleaving local and blocking remote calls can cause performance degradation.

Need to Handle Failures

- In a long-running system, nodes might be down for maintenance.
- Some distributed algorithms use heartbeat protocols and failure detectors to detect the state of the nodes.

Network Partitions and Partial Failures

Network Partition: When two or more servers cannot communicate with each other.

Network partitions can much more troublesome, since independent server-groups can proceed with execution and produce conflicting results.

Cascading Failures

Cascading failures are failures that start in one component and then spread to other components.

Example:

- A server is overloaded and starts responding slowly.
- The clients start timing out and retrying.
- The server gets even more overloaded and starts responding even more slowly.

Distributed Systems Abstractions

We discuss common terms and abstractions used in distributed systems.

Links

Fair-loss link

- A fair loss link is a link that may lose or repeat some packets.
- A sent message can be in the following states from senders' perspective:
    - Not yet delivered
    - Irrecoverably lost
    - Delivered (but the sender doesn't have a way to verify this)

Message acknowledgments

- To get more clarity on message status, we can use acknowledgments to verify that a message has been received.
- We use monotonically increasing message identifiers called "sequence numbers" to identify messages.

An example ACK protocol:

- The sender sends a message M(n), where n is the sequence number.
- The receiver sends an ACK(n) message back to the sender.

When A receives ACK(n), it knows that M(n) has been received by B.

Message retransmits

- Sender keeps sending the message indefinitely.
- This is highly impractical.

Problem with retransmits

- Retransmits are safe only if the message is idempotent.
- If the message is not idempotent, the receiver may end up processing the same message multiple times.
- For example, if the message is "debit $100 from account A", the receiver may end up debiting $100 multiple times.

But:

- Making every message idempotent is not practical.
- So we use deduplication to avoid processing the same message multiple times.

Message order

2 problems:

- Out of order messages
- Duplicate messages due to retransmits

To solve these problems, the receiver can keep a track of the following:

- The last message it received
- The last message it processed

If the receiver receives a message that is older than the last message it processed, it can ignore the message.

If the receiver receives a message that is out of order, it can put the message in a re-ordering buffer and process it later.

This solution gives us a link called "perfect link", and provides following guarantees:

1. Reliable delivery
2. No duplication
3. No messages are created out of thin air

Exactly-once delivery

Most real-world systems don't provide exactly-once delivery. They provide at-least-once delivery == sender retransmits the message until it receives an ACK.

Exactly-once-delivery is possible only if the sender and receiver are aware of each other's state. This is theoretically impossible in a distributed system (Two Generals' Problem).

Two Generals' Problem

This is an experiment that shows that it is impossible to achieve a consensus between two parties in a network where communication is subject to delays and failures.

Challenge: Two generals, each commanding a separate army situated on opposite sides of an enemy city, need to coordinate a synchronized attack. They can only communicate by sending messages through a messenger. The messenger can be captured by the enemy, and the message can be lost.

General A: Sends a message M(n) to General B, where n is the sequence number. General B: Sends an ACK(M(n)) back to General A.

For B to be sure that A has received ACK(M(n)), it has to wait for ACK(ACK(M(n))) from A.

For A to be sure that B has received ACK(ACK(M(n))), it has to wait for ACK(ACK(ACK(M(n)))) from B.

This is an infinite loop. No way for them to coordinate the attack, that is, to reach a consensus.

FLP Impossibility

For a consensus protocol, there are 3 requirements:

1. Agreement: All non-faulty processes must agree on the same value.
2. Validity: The agreed value has to be proposed by some process.
3. Termination: All non-faulty processes must eventually reach the decision state.

FLP Impossibility: Algorithms in asynchronous systems cannot be based on timeouts, and there’s no way for a process to find out whether the other process has crashed or is simply running too slow. The paper shows that, given these assumptions, there exists no protocol that can guarantee consensus in a bounded time. No completely asynchronous consensus algorithm can tolerate the unannounced crash of even a single remote process.

System Synchrony

Problem with asynchronous systems: The assumptions are very pessimistic and impractical.

If we loosen the assumptions, we can build practical synchronous systems. It would assume that processes are progressing at comparable rates, transimission delays are bounded, message delivery cannot take indefinite time.

Defining "synchony" in a distributed system is important since it has a direct impact on the design of the system, and the algorithms that can be used.

Failure Models

We discuss 3 failure models:

1. Crash Faults: We assume that once a process has crashed, it remains in this state.
2. Omission Faults: We assume that the process skips some algorithm steps, or is not able to execute them, or the execution isn't visible to other processes, etc.
3. Arbitrary Faults: We assume that process continues to execute, but it can behave arbitrarily (contradiciting the algorithm steps). Example: A process decides on a value that was not proposed by any process.

Handling Failures

We can mask failures by forming process groups and using redundancy. If a process fails, the other processes can take over, and the user won't notice the failure.

Pending Doubts

NA at the moment.

Things to Read

NA at the moment.

Reading group discussion

NA at the moment.

Chapter 9: Failure Detection

Process related terms:

- dead/failed/crashed: process that has stopped executing its steps completely
- unresponsive/faulty/slow: suspected processes, which may actually be dead

Failure Detector:

- A local subsystem
- Responsible for identifying failed or unreachable processes
- This is done to exclude them from the algorithm
- This guarantees liveness while preserving safety

More terms related to algorithm properties:

- Liveness == guarantees that a specific intended event must occur
- Safety == guarantees that unintended events will not occur
- Completeness == every nonfaulty member should eventually notice the process failure, and the algorithm should eventually reach its final result.

Terms related to algorithm efficiency:

- Accuracy == whether or not the process failure was precisely detected
- Efficiency == how fast the failure detector can identify process failures

We cannot build a failure detector that is both accurate and efficient. These are parameters to tune according to the requirements of the system.

Heartbeats and Pings

Many distributed systems implement failure detectors by using heartbeats.

Basic idea is to find out the state of the remote processes periodically. 2 ways to do this:

- Ping: Send a message to the remote process, and wait for a response. If the response is not received within a specified time period, the process is considered dead.
- Heartbeat: The remote process actively notifies its peers that it's still running by sending messages to them.

Downsides:

- Precision relies on the careful selection of ping frequency and timeout values.
- It does not capture process visibility from the perspective of other processes.

Timeout-Free Failure Detector

State/Components involved:

- Each process maintains a list of neighbors and counters associated with them.
- Processes start by sending heartbeat messages to their neighbors.
- Each message contains the path traveled by the heartbeat so far.

On receiving a heartbeat message, the process:

- increments counters for all participants present in the path.
- appends itself to the path
- sends the heartbeat to the processes that are not present there.

This approach allows us to correctly mark an unreachable process as alive even when the direct link between the two processes is faulty.

Outsourced Heartbeats

- Processes can outsource the task of sending heartbeats to their neighbors.
- This prevents the need for each process to be aware of all other processes in the network. They only need to be aware of a subset of connected peers.

Phi-Accural Failure Detector

- Instead of using a binary scale (0/1), phi-accrual failure detector uses a continuous scale, capturing the probability of the monitored process's crash.
- It collects arrival times of the most recent heartbeats from the peer processes.
- It uses the collected information to make a reliable judgment about node health.

φ = phi == how likely we can make correct decision about a process's liveness == how likely we can make a mistake and receive a heartbeat that will contradict the calculated assumptions.

Gossip and Failure Detection

- Each process maintains 3 things:
  - a list of neighbors
  - their heartbeat counters
  - timestamps specifying when the heartbeat counters were last incremented
- Each process periodically selects a random neighbor and sends it a heartbeat message.
- The receiving process increments the counter for the sender in his list of neighbors.

If any node did not update its heartbeat counter for a long time, it is considered dead.

Reversing Failure Detection Problem Statement

Problem so far:

- Propagating information about failures is not always possible.
- Propagating it by notifying all processes is expensive / not efficient.

FUSE is one of the approaches to implement a failure notification service.

- It focuses on reliable and cheap failure propagation.
- Works even when the network is partitioned.

How it works?

- We arrange all active processes in groups.
- If one of the groups becomes unavailable, all participants detect the failure. In other words, if a single process failure is detected, it is converted and propagated as a group failure.

This approach allows detecting failures in the presence of any pattern of disconnects, network partitions, and node failures.

Within the group:

- Processes send ping messages to each other periodically.
- If a process does not receive a ping message from a peer for a long time, it stops responding to ping messages itself.

Pending Doubts

NA at the moment.

Things to Read

NA at the moment.

Reading group discussion

NA at the moment.

Chapter 10: Leader Election

Problem: Synchronization can be costly because of significant communication overhead.

Solution: Leader election.

Leader == a process that is responsible for coordinating the actions of the other processes.

Liveness of election algorithm

- Guarantees that most of the time, there will be a leader in the system.
- Election will eventually terminate and a leader will be elected.

Safety:

- Ideally, only one process should be elected as the leader.

When is leader election triggered?

- When the system initializes.
- When the existing leader fails.

Some algorithms (ZAB, Multi-Paxos, Raft) use temporary leaders to reduce the number of messages required to reach consensus.

Bully Algorithm

- Leader election algorithm.
- Uses process ranks to elect a leader.
    - Each process is assigned a unique rank.
    - During the election, the process with the highest rank is elected as the leader.
- Called "bully" because the process with the highest rank "bullies" the other processes into accepting it as the leader.
- Also called monarchial leader election.

Election happens when:

- one of the processes notices that there's no leader. (it was never initialized)
- existing leader has stopped responding to requests.

Election happens in 3 steps:

1. Process (that noticed the absence of a leader) sends election messages to all processes with higher ranks.
2. If no response is received, the process declares itself as the leader, and notifies all other lower ranked processes.
3. If a response is received, the process notifies the highest-ranked process that responded to do step 2.

How to write a nested numbered list in markdown?

1. Step 1
    1. Step 1.1
    2. Step 1.2

Problems:

1. Safety: In the presence of network partitions, multiple leaders can be elected.
    1. We can end in a situation where nodes get split into independent subnets, and each subnet elects its own leader. This situation is called a split brain.
2. Strong preference for higher ranked processes. This can lead to a situation if the higher-ranked process keeps failing shortly after winning the election.

Next-In-Line Failover

- Each elected leader provides a list of failover candidate nodes.
- When a process detects that the leader has failed, it starts a new election round by contacting the highest-ranked failover candidate from the list.
- If the failover candidate is not available, the process contacts the next highest-ranked candidate. If it is available, it becomes the new leader.
- If the process itself is the highest-ranked candidate, it becomes the new leader and notifies all other processes.

This improves upon the Bully algorithm by reducing the number of messages required to elect a new leader.

Candidate/Ordinary Optimization

This is another optimization to reduce the number of messages required to elect a new leader.

In this, we split the nodes into 2 subsets:

1. Candidate nodes: These are the nodes that can become the leader.
2. Ordinary nodes

Process: Ordinary process initiates an election round by sending an election message to all candidate nodes, picks the highest-ranked candidate that responds, and notifies all other processes.

Invitation Algorithm

- Invitation Algorithm allows processes to invite other processes to join their groups.
- By definition, it allows multiple leaders to coexist in the system.

✨ **Side learning:** The Invitation Algorithm provides a protocol for forming groups of available processors within partitions, and then creating larger groups as failed processors are returned to service or network

partitions are rectified. (Source)

Steps:

- Each process starts as a leader of a new group, which contains only itself.
- Group leaders contact peers to invite them to join their groups.
    - If the peer process is a leader, both the groups are merged.
    - If the peer process is not a leader, it responds with the group-leader-ID, allowing communication between the two group leaders, and leading to merging of the groups.

Generally, leader of a larger group becomes the leader of the merged group. This way, the number of messages required to elect a leader is reduced, as processes from the smaller group have to be notified about the new leader.

Ring Algorithm

- All nodes are arranged in a ring.
- All nodes are aware of their neighbors (predecessor and successor).

Steps:

- Each node starts an election round by sending an election message to its successor.
- If the successor is not available, the node skips it and sends the message to the next node, until it finds a node that responds.
- Nodes keep collecting the "live node set". They add themselves to the set before passing it to their successor.
- The whole ring is traversed once, until the initiator receives the message back with the "live node set".
- When the message reaches the initiator, the highest-ranked node in the "live node set" is elected as the leader.
- The initiator notifies all other nodes about the new leader.

Pending Doubts

1. It was mentioned that some algorithms (ZAB, Multi-Paxos, Raft) use temporary leaders to reduce the number of messages required to reach consensus. Does this simply mean they re-elect a leader every now and then? If yes, how does it help?

[Update] Got a couple of answers by folks in the reading group discussion thread:

Answer 1: I think we'll get to Raft in another chapter, but yes, they do elect a new leader every so often. A leader (A) won't itself step down until it sees another node (B) claiming to be a leader for a more recent election than (A) is aware of. A follower won't attempt to become a leader until it hasn't heard from that leader within a period of time. The leader is responsible for sending out periodic heartbeats to attempt to avoid followers attempting to become leaders.

Answer 2: Contributing a bit to answer 1 - By utilizing the leader, the algorithms you mention put more pressure on one node but can reduce the number of messages exchanged to reach consensus. The leader decides the sequence number and replicates that, so it does not need additional round-trips between the members. There are also a few leaderless (or, in contrast, multi-leader) algorithms out there. Each employs different techniques to reduce the round-trip of messages to deal with no dedicated node ordering

everything and usually has a worst-case for contention. I believe we'll look at some of that in the following chapters.

2. In Candidate/Ordinary Optimization, can the candidate nodes not initiate an election round? If yes, why not?

[Update] Got an answer by someone in the reading group discussion thread:

From my understanding, the candidates never initiate an election. All the responsibility is on the ordinary set. I believe an election does not start if the FD doesn't suspect a leader, and the delta parameter would help in false-positive cases. With the candidates acting reactively to ordinary requests, the number of false positives also reduces since it has fewer nodes starting the election. This is only an assumption on my part, but it seems a trade-off. A more stable algorithm, but it would take longer to identify and replace a failed leader.

Things to Read

NA at the moment.

Reading group discussion

NA at the moment.

Chapter 11: Replication and Consistency

Consistency models explain the visibility semantics and system behavior in the presence of multiple copies of data.

Fault tolerance == ability to continue operating despite the failure of some of the components. This is achieved by replicating data across multiple nodes.

Data replication == Introduction of multiple copies of data across multiple nodes, so that the system can continue to operate even if some of the nodes fail.

If we have data replicated across multiple nodes, we need to ensure that the data is consistent across all the nodes. Data can go inconsistent since every write has to be propagated to all the nodes.

Achieving Availability

Aim: high availability, low downtime.

Solution: Introduce redundancy and replication.

New problem (based on the above solution): How to keep the replicas consistent?

Infamous CAP

CAP == Consistency, Availability, Partition tolerance.

Availability == ability of the system to serve a response for every request successfully.

Partition tolerance == ability of the system to continue operating despite a network partition in the system (nodes divided into multiple subnets).

CAP theorem == It is impossible for a distributed system to simultaneously provide all three guarantees (consistency, availability, partition tolerance).

Partition tolerance is unavoidable in a distributed system. So, we have to choose between consistency and availability == either (Consistency + Partition tolerance) or (Availability + Partition tolerance).

✨ **Side learning:** Good resource for intuitive understanding of why both Consistency and Availability can't be achieved simultaneously in the presence of a network partition: Arpit Bhayani's video on CAP Theorem

Instead of providing a binary choice between consistency and availability, we can provide relaxed guarantees for both. We can define 2 tunable metrics - Harvest and Yield.

Harvest and Yield:

- Harvest:How complete is the data that is returned? (example: 99 rows being returned instead of 100 rows, 100 being the expected number of rows)
- Yield: Number of requests that were completed successfully, compared to the total number of attempted requests.

Shared Memory

From client's perspective, the system appears to be a single machine with a single shared memory.

Register == a single unit of storage that can be read and written to.

Shared memory in distributed systems == an array of such registers.

3 types of registers:

1. Safe registers: Reads may return arbitrary values within the range of values being concurrently written to the register.
2. Regular registers: Reads can return only the value written by the most recent completed write, or the value being written by the current overlapping write operation.
3. Atomic registers: These guarantee linearizability. Everyw rite operation has a single point in time:
    1. before which every read operation returns an old value
    2. after which every read operation returns a new value.

✨ **Side learning:** This StackOverflow answer explains the difference in a very nice way with excellent example: Link

Ordering

In a distributed system with, it's unclear what would be the outcome of a read(x) operation when there are concurrent read(x) and write(x) operations. It becomes even more complicated when there are replicated copies of data involved.

We define "consistency models" to explain the visibility semantics and system behavior in such situations.

Consistency Models

We need models to define the semantics in the face of concurrent operations.

Consistency models:

- provide semantics and guarantees.
- describes what expectations the clients should have about the system behavior.
- describes which state invariants are acceptable --- OR --- describes "allowable relationships" between copies of data placed on replicas.

Strict Consistency

- Any write by any process is instantly available for subsequent reads by any process.
- This is just theoretical. It's not possible to achieve this in practice.

Linearizability

- Linearizability is the strongest single-object, single-operation consistency model.
- Writes becomes visible to all readers "exactly once" between the invocation and response of a write operation --- AND --- no client can observe intermediate state of a write operation.
- Effects become visible in a way that makes them appear sequential.

Linearization point: The point in time at which a write operation becomes visible to all readers.

Cost of linearizability:

- Linearizability is expensive to implement. It requires coordination between replicas.
- In concurrent programming: Achievable using compare-and-swap (CAS) operations.
- In distributed systems: Requires coordination and ordering. Achievable using consensus protocols.

Sequential Consistency

Achieving linearizability is expensive. So, we relax the guarantees to achieve sequential consistency which still provides rather strong consistency guarantees.

- Sequential consistency: Allows ordering operations as if they were executed in some sequential order, but in the same order for all processes.

Different from linearizability in the following way: Under linearizability, the operation has to become visible within the operation duration. Under sequential consistency, the operation can become visible after the operation duration.

Causal Consistency

Under this model:

- all processes have to see causally related operations in the same order.
- concurrent writes with no causal relationship can be observed in different orders on different processes.

In a causally consistent system, we get session guarantees for the application: monotonic reads and writes, read-your-writes, writes-follow-reads.

Implementation: Via logical clocks like vector clocks.

Vector clocks: Vector clock is a structure for establishing the partial ordering of events in a distributed system, detecting causality violations, and resolving divergence between the event chains.

- Processes maintain vectors of logical clocks, with one clock per process.
- Every clock starts at the initial value.
- The value of a clock is incremented every time a new event occurs (for example, a write occurs).
- On receiving a vector clock from another process, a process updates its local vector to the highest clock values per process from the received vectors.

✨ **Side learning:** Vector clocks allow us to detect whether 2 events are:

1. causally related: If the vector clock of one event (V(e1)) is less than or equal to the vector clock of the other event (V(e2)). That is, `V(e1) <= V(e2)`.
2. concurrent: If `NOT (V(e1) <= V(e2)) AND NOT (V(e2) <= V(e1))`.

Source: Blog on Vector Clocks

Session Models

Session models == thinking about the DB state from client's perspective.

- Client should be able to reason about the state of the database based on the operations it has performed.
- Client should read its own writes. (read-own-writes)

Eventual Consistency

Eventual consistency == updates propagate through the system asynchronously. The system eventually reaches a consistent state.

Tunable Consistency

Eventually consistent systems usually implement tunable consistency models, using 3 variables:

1. Replication factor N: Number of nodes that will store a copy of data.
2. Write consistency W: Number of nodes that must acknowledge a write before it is considered successful.
3. Read consistency R: Number of nodes that have to respond to a read operation to succeed.

For example:

- If the system is using a consistency model having `(R + W > N)`, then the system guarantees that the read operation will always return the most recent write.
- This is because there's always an overlap between the nodes that are used for reading and writing.

Other aspects:

- Write-heavy systems can use `W = 1` and `R = N`.
- Read-heavy systems can use `W = N` and `R = 1`.

Quorums: A consistency level having `[N/2] + 1` nodes. Or simply put, a majority of nodes.

Witness Replicas

Problem: Using replication increases storage costs because we need to store multiple copies of data.

Solution: Use witness replicas.

- Instead of storing copies of data on each replica, we split replicas into "copy replicas" and "witness replicas".
- Copy replicas store the actual data.
- Under normal situations, witness replicas store only a record indicating that the write has been performed.
- If number of copy replicas falls below a threshold (cannot achieve quorum), witness replicas can be promoted to copy replicas, and temporarily store the actual data.
- When the copy replicas recover, the witness replicas can be demoted back to witness replicas, OR the recovered replicas can become witness replicas.

Witness replicas are used in popular databases like Cassandra and Spanner.

Strong Eventual Consistency and CRDTs

Strong eventual consistency:

- It is a middle group between eventual consistency and linearizability.
- This allows relaxed consistency guarantees, but not as much as eventual consistency.
- Under this model, updates are allowed to propagate late or out of order. But when all updates are propagated, conflicts between them are resolved and they are merged to form a consistent state.
- Operations need to preserve additional state that can be used to resolve conflicts. Example: Using CRDTs.

CRDTs:

- Conflict-free Replicated Data Types.
- These are special data structures that are replicated across multiple nodes.
- CRDTs are useful in eventually consistent systems, since it allows the replica states to temporarily diverge.
- CRDTs allow us to reconstruct the state of the system by merging the states of all the replicas.

Simplest example of CRDT: Operation-based Commutaive Replicated Data Types (CmRDTs). For CmRDTs to work, the operations need to be:

1. Side-effect free.
2. Commutative: It shouldn't matter whether x is merged into y or y is merged into x.
3. Causally ordered

Pending Doubts

NA at the moment.

Things to Read

NA at the moment.

Reading group discussion

1. A real life example of consistency came up at work in the context of a batch job that has multiple readers pulling from a GCS bucket: https://cloud.google.com/storage/docs/consistency

Chapter 12: Anti-Entropy and Dissemination

More than the data records, we want the cluster-wide metadata changes to be propagated to all the nodes in the cluster quickly and reliably. This chapter discusses the anti-entropy and dissemination protocols that are used to achieve this.

Cluster-wide metadata: Which nodes are joining or leaving, node states, failures, schema changes, etc

3 ways:

1. Notification broadcast from one process to all other processes
2. Periodic peer-to-peer information exchange
3. Cooperative broadcast, where receivers help forward the message to other nodes

Entropy == measure of disorder in the system == message of state divergence between nodes (in distributed systems context)

Anti-entropy == attempting to lower the convergency time bounds in eventually consistency systems.

Anti-entropy measures can be of 2 ways:

1. Background processes: Using structures like Merkle trees to detect divergence and then using a background process to reconcile the differences
2. Foreground processes: Piggybacking on the normal read/write operations to detect divergence and reconcile the differences. Example: hinted handoff, read-repair, etc

Read Repair

- Detects divergence during read operations
- Coordinator node sends read request to all replicas. If replicas send back different values, then the coordinator node picks the most recent value and sends it to all replicas which reported older values.

2 types of read repair:

1. Blocking read repair: Client request has to wait until the coordinator node has repaired all the replicas == we sacrifice availability for consistency
2. Asynchronous read repair: Client request does not have to wait for the coordinator node to repair all the replicas, coordinator does that in the background asynchronously == we sacrifice consistency for availability

Digest Reads

This is an optimization to the read repair process.

- Coordinator issues only one full read request to one replica, and issues "digest requests" to all other replicas.
- Digest request returns a hash of the data (instead of the actual data), which is much smaller than the actual data.
- Coordinator compares the hash values from all replicas.
    - If they are the same, then the replicas are in sync.
    - If they are different, then the coordinator issues a full read repair request to all replicas which reported a different hash value.

Hinted Handoff

- This is a write-repair mechanism.
- If a node fails to acknowledge a write request, then a special record (called hint) is stored by either the coordinator or by one of the replicas. This hint is then "replayed" to the failed node when it comes back online.

Merkle Trees

Problem with read-repair: They only fix inconsistencies on the data that was queried. We still need a background process to fix inconsistencies on the data that is not being queried.

We need an efficient way to detect inconsistencies in the data. Merkle trees are one such way.

Merkle trees:

- They are composed of a hashed representation of the data, structured in the form of a tree of hashes.
- Lowest level of the tree contains the hashes of the data records.
- Higher levels of the tree contain the hashes of the lower-level hashes.

Good aspect: It's efficient to detect inconsistencies in the data, by comparing the hashes of the data records (going from the root node to the leaf nodes).

Bad aspect: Any write operation would cause the hashes to be re-computed recursively for the entire subtree. This is expensive.

Bitmap Version Vectors

- Each node keeps a per-peer log of operations that have occurred locally or were replicated.
- During anti-entropy, logs are compared, and missing data is replicated to the target node.

Gossip Dissemination

Objective: To spread information from one process to the rest of the cluster.

Some terms:

- Infective process: The process that holds the information to be spread.
- Susceptible process: The process that has not yet received the information.
- Removed process: The infected process that are not spreading information anymore after a period of spreading the information.

Since we're just spreading information randomly, and relying on probabilities for the information to reach all nodes of the cluster, we need a metric that helps us figure out when we can stop spreading the information. This is done using "loss of interest" function == the degree with which we've lost interest in spreading the information.

Gossip Mechanics

Message redundancy == metric that measures the overhead incurred due to repeated delivery. Redundancy is inevitable since it's probabilistic.

Latency: Amount of time it takes for the system to reach convergence. This is not the same as the time it takes for the information to reach all nodes. This is because information would still continue to spread (for some time) even after all nodes have received the information.

- Processes periodically select `f` peers at random. `f` is a configurable parameter called fanout.
- Then they share the information with the selected peers.
- Over time, nodes notice that they are receiving the same information multiple times.
- "Interest loss" is calculated using multiple approaches:
    - probabilistically (computing probability of stopping the propagation for each process on every step)
    - using a fixed threshold (counting the number of received duplicates, and stopping when the threshold exceeds a certain value)

Overlay Networks

Problem with random selection of nodes: It involves a lot of redundant communication.

Solution: We can reduce this by using a structured overlay network.

- We construct an overlay network of nodes, and then use the overlay network to spread information.
- Nodes in the system can form "spanning trees".
- Information is then spread using the spanning trees.

Hybrid Gossip

- Push/lazy-push multicast trees (also called Plumtrees) make a trade-off between the overhead of gossip and the latency of spanning trees.
- Nodes send full messages to a subset of their peers, and send only the "message ID" to the rest of the peers.
- If the receiving node has not seen the message before (corresponding to the message ID), then it requests the full message from its peers.

Partial Views

Broadcasting messages to all known peers is expensive. We can reduce this by using partial views.

- We maintain a partial view of the cluster, and only send messages to the nodes in the partial view.
- The partial view is periodically refreshed using gossip.

Example: Hybrid Partial View (HyParView) protocol.

✨ **Side learning:**

Really good article that explains HyParView: https://www.bartoszsypytkowski.com/hyparview/

Pending Doubts

NA at the moment.

Things to Read

NA at the moment.

Reading group discussion

NA at the moment.

Chapter 13: Distributed Transactions

- Distributed transactions are a way to ensure that a set of operations are performed atomically across multiple nodes in a cluster.
- Transaction atomicity == Either all the results of a transaction become visible, or none of them become visible.
- To ensure atomicity, transactions need to be recoverable. (so that they can be rolled back if they fail)

Making Operations Appear Atomic

To make multiple operations appear atomic, we use algorithms belonging to "atomic commitment" category.

Atomic Commitment:

- A transaction will not commit if any of the participants vote against it.
- We discuss two algorithms for atomic commitment:
  - Two-phase commit (2PC)
  - Three-phase commit (3PC)

Two-Phase Commit

- Two-phase commit (also known as 2PC) has 2 phases.
- Phase 1: Prepare
  - Coordinator notifies all participants about the new transaction by sending the proposed value.
  - Participants respond/vote with a yes/no answer.
  - If any participant says no, the coordinator aborts the transaction.
  - If all participants say yes, the coordinator moves to phase 2.
- Phase 2: Commit/Abort
  - Coordinator sends a commit request to all the participants.
  - Participants respond with an acknowledgement.
  - If any participant fails to respond, the coordinator aborts the transaction.
  - If all participants respond, the coordinator commits the transaction.

Cohort Failures in 2PC

Following cases:

1. Cohort fails before voting: Coordinator will abort the transaction, as it needs all votes to be positive.
2. Cohort fails after voting:

- If it had voted "YES": When it recovers, it has to find out the actual outcome of the vote before serving values correctly, as the coordinator might have aborted the commit due to other cohorts' decisions.
- If it had voted "NO": It doesn't have to do anything, since it wouldn't have had precommitted the transaction anyway, so it can release any locks it had acquired.

Coordinator Failures in 2PC

Following cases:

1. Coordinator fails before proposing a value: Nothing needs to be "fixed" as the transaction was never proposed to the cohorts.
2. Coordinator fails after collecting votes:

- If none of the cohorts have received any result from coordinator: Cohorts are left blocked. This is called the "blocking" issue with 2PC.
- If a subset of the cohorts have not received the result: They contact peers, since the decision (if received) would be unanimous.

Databases that use 2PC: MySQL, PostgreSQL, MongoDB, etc.

✨ **Side learning 1:** An AWESOME blog which explains the failure cases of 2PC in a very detailed way: Link to blog

I highly recommend checking out the above blog. It helped me understand the different failure cases of 2PC in a much better way.

✨ **Side learning 2:** I was confused on how or why PostgreSQL would need 2PC as that's meant for distributed transactions. Here's why:

- https://www.endpointdev.com/blog/2006/05/postgresql-supports-two-phase-commit/
- https://www.postgresql.org/docs/10/sql-prepare-transaction.html

Three-Phase Commit

Problems with 2PC: The "blocking" issue.

Solution: Introduce timeouts in 2PC.

Problem with introducing timeouts: The "inconsistency" issue.

What is the "inconsistency" issue? If the coordinator goes down when it receives a failure from a participant and intends to send an abort request to other participants, the corresponding distributed transaction is supposed to fail. However, some participants may commit it due to timeout.

Solution: Introduce a third phase in 2PC.

3 phases of 3PC:

- Phase 1: Propose
  - Coordinator sends a proposed value to all participants, and collects their votes.
  - If any participant votes no, the coordinator aborts the transaction.
  - If all participants vote yes, the coordinator moves to phase 2.
- Phase 2: Prepare
  - Coordinator notifies participants about the vote results.
  - If any participant votes no, the coordinator aborts the transaction by sending an abort request to all participants.
  - If all participants vote yes, the coordinator sends a "prepare" request to all participants. Think of "prepare" request as coordinator telling the participants that everyone has voted yes, so you should prepare yourself to commit the transaction. Next, we move to phase 3.
- Phase 3: Commit/Abort
  - If the coordinator receives a "prepared" acknowledgement from all participants, it sends a "commit" request to all participants. If the coordinator fails before sending the commit request, the participants will commit the transaction by themselves after the timeout as they are aware that everyone has voted yes.
  - If any participant fails to respond with the "prepared" acknowledgement, the coordinator aborts the transaction by sending an abort request to all participants. If the coordinator fails before sending the abort request, the participants will abort the transaction by themselves after the timeout.

Coordinator Failures in 3PC

- The worst-case scenario for 3PC is a network partition.
- In this case, the coordinator and some of the participants are on one side of the partition, and the remaining participants are on the other side.
- The coordinator and the participants on one side of the partition can communicate with each other, but they cannot communicate with the participants on the other side of the partition.
- This results in a split-brain situation, where the participants on one side of the partition proceed with a commit, while the participants on the other side of the partition abort the transaction.
- This leaves the system in an inconsistent state.

Why is 3PC not used in practice?

- Larger message overhead than 2PC.
- Does not work well in the presence of network partitions.

✨ **Side learning:**

- This StackOverflow answer explains the difference between 2PC and 3PC in a very good way: https://stackoverflow.com/a/55834191
- This blog by TiKV is also good: https://tikv.org/deep-dive/distributed-transaction/distributed-algorithms/

Distributed Transactions with Calvin

- Calvin is a transaction scheduling system in distributed systems.

- It has 4 components, in order of a transaction lifecycle:
    - Sequencer: The sequencer determines the order in which transactions are executed, and establishes a global transaction input sequence.
    - Scheduler: The scheduler is responsible for scheduling transactions for execution.
    - Worker: The worker thread is managed by the scheduler, and is responsible for executing a transaction.
    - Storage: Worker threads read and write data from the storage layer.

✨ **Side learning:**

- The core idea of calvin is as follows: "When multiple machines need to agree on how to handle a particular transaction, they do it outside of transactional boundaries — that is, before they acquire locks and begin executing the transaction."
- Source: https://medium.com/@balrajasubbiah/calvin-fast-distributed-transactions-for-partitioned-database-systems-17b1aee15e09

I feel the above blog is an easier-to-understand text than the book itself. I highly recommend reading the above blog.

Distributed Transactions with Spanner

- Spanner is a distributed database system developed by Google.
- It uses 2PC on consensus groups per partition/shard.

Spanner offers 3 operation types:

1. Read-only transactions: These are lock-free transactions that can be executed on any replica.
2. Read-write transactions: These require locks, pessimistic concurrency control, and presence of leader replica.
3. Snapshot reads: These are lock-free reads that can be executed on any replica.

As mentioned above, Spanner uses 2PC on "consensus groups per partition/shard". Why?

- 2PC algorithm requires presence of all participants to commit a transaction. This hurts availability of the system.
- Solution: Spanner uses "Paxos groups" instead of individual nodes as the participants in 2PC. This allows 2PC to continue even if some members of the Paxos group are down. This improves availability of the system.

Paxos is covered in greater detail in Chapter 14, so hopefully this would become more intuitive after reading that chapter.

Database Partitioning

Partitioning: A logical division of data into smaller segments.

Example:

- A database with 1000 users can be partitioned into 10 partitions, each containing 100 users.
- A database with 1000 locations can be partitioned on the zip code.

How do partitions work?

- The individual segments can reside on different replica sets. This is called "sharding". Each replica set acts as a source of truth for a subset of the data.
- Database systems have to store a routing key, which is used to determine which partition a read/write request should be routed to. This is generally done by mapping a hash value to a node ID (which is essentially the partition).

Problem: Adding and removing nodes from a cluster would require rebalancing of the data (depending on the partitioning scheme / hash function used).

Solution:

- Use consistent hashing.
- Each partition / node is assigned a hash value which is mapped to a "ring" of hash values.
- This helps because a change in the number of partitions / nodes would only affect the partitions / nodes immediately before and after the changed partition / node.

Distributed Transactions with Percolator

- Percolator is a library that implements a transactional API on top of the distributed database Bigtable.
- Each transaction has to consult a clusterwise timestamp oracle twice:
    - to get timestamp for start of transaction
    - to get timestamp during commit
- Writes are buffered and committed using client-driven 2PC.

Coordination Avoidance

Invariant Confluence: Property of a system that ensures that 2 different states --- which adhere to invariant rules, but are diverged --- can be merged into a single state, and the invariant rules would still hold in the merged state.

RAMP (Read-Atomic Multi Partition)

- RAMP transactions use MVCC and metadata of in-flight operations to fetch missing updates from other nodes.
- Writes in RAMP are made visible using 2PC.

Pending Doubts

NA at the moment.

Things to Read

NA at the moment.

Reading group discussion

NA at the moment.

Chapter 14: Consensus

Consensus == Multiple processes reaching agreement on a value.

Correct process == A process that hasn't crashed and is executing its algorithm.

3 properties of consensus algorithms:

1. Agreement: All correct processes decide on the same value.
2. Validity: The value decided upon is proposed by some process.
3. Termination: All correct processes eventually decide on a value.

Broadcast

Broadcast == A process sends a message to all other processes.

- Reliable broadcast == All correct processes receive the message.
- Naive version of reliable broadcast == All processes send the message to all other processes.
- Downside of naive version:
    - Network is flooded with messages, since this requires N^2 messages, where N is the number of processes.
    - Order of messages is not guaranteed.

Atomic Broadcast

Atomic broadcast guarantees 2 properties:

- Atomicity: Either all correct processes deliver the message, or none do.
- Order: If a correct process delivers message m1 before message m2, then all correct processes deliver m1 before m2.

Virtual Synchrony

Atomic broadcast delivers messages to a static group of processes. Virtual synchrony extends this to dynamic groups.

Virtual synchrony:

- Ordered Message Delivery: Delivers totally ordered messages consistently to all group members.
- Group Organization: Processes organized into groups with shared views, ensuring consistent order.
- Dynamic Group Changes: Group views change with join, leave, or failure announcements, affecting message association.
- Atomic Delivery Guarantees: Ensures atomic message delivery between view changes, serving as barriers. Limited adoption in end-user systems.

Zookeeper Atomic Broadcast (ZAB)

ZAB is a popular implementation of atomic broadcast. It is used in Zookeeper, a hierarchical distributed key-value store.

Processes in ZAB can take one of the following roles:

1. Leader: Drives the process, establishes event order, and manages algorithm execution in ZAB.
2. Follower: Listens to the leader, follows broadcasted messages.

After leader is established, protocol executes in 3 phases:

1. Discovery: Leader learns the latest epoch from followers, proposes a new epoch, and obtains follower responses regarding the latest transaction in the previous epoch.
2. Synchronization: Leader proposes itself as a leader, collects acknowledgments from followers, and ensures synchronization, delivering committed proposals from previous leaders before any new epoch proposals.
3. Broadcast: Leader initiates active messaging, establishes message order, and broadcasts client messages by proposing, waiting for follower acknowledgments, and committing.

Advantages of ZAB:

- Efficiency: Broadcast processes requires only 2 rounds of messages.
- Fault Tolerance: Leader failure is handled by electing a new leader.

Paxos

Paxos == Most widely known consensus algorithm.

Participants in Paxos can take the following roles:

1. Proposer: Proposes a value to acceptors and collects votes.
2. Acceptor: Votes for a value proposed by a proposer.
3. Learner: Takes the role of replicas, and stores the outcomes of accepted proposals.

Paxos Algorithm

Paxos algorithm executes in 2 phases:

1. Propose/Voting Phase: Proposer sends a proposal to acceptors, who vote for the proposal. If a proposal receives a majority of votes, it is accepted.
2. Replication Phase: Proposer distributes the accepted value to acceptors.

Acceptor has to follow these invariants when responding to a proposal with Prepare(n) message, where n is the proposal number:

1. If acceptor has not responded to any Prepare(m) message (where m > n), it promises not to respond to any proposal with a number less than n.
2. If acceptor has already accepted any other proposal having proposal number m earlier, it notifies the proposer that it has already accepted a proposal with a proposal number m.
3. If acceptor has already responded to Prepare(m) message (where m > n), it notifies the proposer about the existence of a higher proposal number.
4. Acceptor can respond to more than one Prepare(x), as long as the later proposals have a higher proposal number.

✨ **Side learning:** Very good explanation of Paxos - https://www.youtube.com/watch?v=SRsK-ZXTeZ0

Quorums in Paxos

Quorum == The minimum number of votes required for the operation to be performed.

Generally, quorum == majority of participants.

Once sufficient number of participants (quorum) have accepted the proposal, it's guaranteed that the value will be accepted, since any two majorities will have at least one participant in common.

To guarantee "liveness", the protocol requires 2f+1 participants, where f is the number of failures that can be tolerated.

Failure Scenarios

Case 1: Proposer fails during 2nd phase, before broadcasting value to all acceptors

A new proposer would pick up the value from quorum and broadcast it to all acceptors.

- If the value received from quorum is the same as proposed by original proposer, the new proposer will continue with the broadcast using the same value.
- If the value received from quorum is different from the one proposed by original proposer (and the new proposer doesn't know this), the new proposer will commit the value received from quorum, which is not the value proposed by original proposer.

Case 2: When 2 or more proposers start competing

- To avoid multiple proposers failing to collect a majority, a random backoff time can be used for each proposer to wait.

TLDR: Paxos can tolerate acceptor failures, but only if there are still enough acceptors alive to form a majority.

Multi-Paxos

- Problem with Paxos: It requires 2 phases for every proposal, which can be slow.
- Solution: Multi-Paxos, where a single proposer can propose multiple values in a single round.
- To avoid repeating the "propose" phase, multi-paxos has a leader (effectively a distinguished proposer) that can propose values without going through the "prepare" phase.

Fast Paxos

- Fast Paxos is an optimization of the original Paxos algorithm designed to reduce the number of message rounds required for reaching consensus.
- This is done using "fast rounds", when acceptors can proceed with messages from proposers other than the established leader.

✨ **Side learning:** Good short blog on Fast Paxos

Egalitarian Paxos

Problems with distinguished proposer as a leader:

- Makes system prone to leader failures.

- Dispropotionate load on leader, impairing system performance.

Solution:

- Instead of using leader and proposal numbers to order proposals, we use a leader that is responsible for the commit of the specific command, but not for the ordering of the commands.
- The event order is established by resolving dependencies between submitted messages.

"Dependencies" == All commands that interfere with a current proposal.

Flexible Paxos

- Instead of requiring quorum for both phases, Flexible Paxos relaxes the quorum requirements.
- It only requires quorum for the 1st phase (voting) to intersect with a quorum for the 2nd phase (replication).
- Flexible Paxos allows trading availability for latency.
    - We reduce the number of nodes required for the 2nd phase, which reduces the latency.
    - But we have to collect more votes in the 1st phase, requiring more participants to be available in the 1st phase.

For example:

1. Let's say we have 5 acceptors.
2. Let's say we require collecting votes from 4 acceptors to accept a proposal.
3. Then we only need to wait for responses from 2 nodes during the replication phase, since it's guaranteed that atleast one of the nodes that voted for the proposal will be present in the 2 nodes that we are waiting for.

Generalized Solution to Consensus

I feel this section is a bit too theoretical, and is rushed in the book. So instead of trying to create notes, I'd recommend checking out the following blog: https://blog.acolyer.org/2019/03/08/a-generalised-solution-to-distributed-consensus/

Raft

Raft == A consensus algorithm designed to be easy to understand.

Participants in Raft can take one of the following roles:

1. Candidate: Proposes itself as a leader.
2. Leader: Temporary leader, that handles client requests and replication.
    - Leader is elected for a period called "term".
    - "Term" is identified by a monotonic increasing number.
3. Follower: Responds to requests from leaders and candidates. Persists log entries.

How is global partial order maintained in Raft?

- Time is divided into "terms" / "epochs".
- Each command is uniquely identified by 2 things:
    1. the term number

    2. an index (message number) within the term.

Main components of Raft algorithm:

1. Leader Election:
    1. Candidate P1 initiates by sending RequestVote with term, last known term, and last log entry observed.
    2. Majority votes lead to successful election, ensuring each process can vote for at most one candidate.
2. Periodic Heartbeats:
    1. Leader sends periodic heartbeats to followers for liveness.
    2. If a follower doesn't receive heartbeats within an election timeout, it assumes leader failure and triggers a new election.
3. Log Replication/Broadcast:
    1. Leader appends new log entries using AppendEntries messages.
    2. Message includes leader’s term, index, term of the preceding log entry, and one or more entries for replication.

Leader Role in Raft

- Election Criteria:
    - Leader elected from nodes with all committed entries.
    - Follower denies the vote if its log is more up-to-date.
- Leadership:
    - Leader replicates client requests, appends entries to log.
- Follower Response:
    - Followers append entries, lets leader know that it was persisted.
    - As soon as enough replicas send their acknowledgments, leader commits the entry.
- Log Flow:
    - Unidirectional flow from leader to follower, because only the most up-to-date candidate can be elected as leader.

Failure Scenarios

Scenarios:

- Split Vote: Multiple followers decide to become candidates, and no candidate gets majority votes.
    - Randomized timers are used to reduce probability of split vote.
- Followers may be down or slow to respond, impacting message delivery.
    - Leader retries message delivery and optimizes performance by sending multiple messages in parallel.
- Leader may fail, and a new leader may be elected.
    - Newly elected leader restores cluster state to the last known up-to-date log entry.

Byzantine Consensus

All algorithms discussed so far assume that participants are honest. Byzantine consensus algorithms are designed to tolerate malicious participants.

Byzantine algorithms require extensive N^2 messages for cross-validation.

- This is because nodes cannot rely on the messages from other nodes, and have to validate the received message by comparing it with the message received by majority of the nodes.

PBFT Algorithm

Practical Byzantine Fault Tolerance (PBFT): It is a Byzantine consensus algorithm that tolerates f failures in a network of 3f+1 nodes.

Cluster Configurations with "Views"

- Views distinguish between configurations.
- Each view has a "primary" and remaining nodes are "backups".
- Primary executes client operations, broadcasting requests to backups.
- Client waits for 2f + 1 matching responses from backups for any operation to succeed.

Protocol Phases

- Pre-prepare: Primary node broadcasts view ID, unique identifier, payload, and payload digest.
- Prepare: Backups broadcast Prepare messages with view ID, message ID, and payload digest, validating against pre-prepare.
- Commit: Backups broadcast Commit messages, waiting for 2f + 1 matching Commit messages for execution.

Recovery and Checkpointing

- Replicas store messages until executed by at least f + 1 nodes.
- Nodes compute a digest for state verification during recovery.
- Primary makes stable checkpoints after N requests, waits for 2f + 1 replica responses.

Pending Doubts

NA at the moment.

Things to Read

NA at the moment.

Reading group discussion

NA at the moment.

Key Nuances

- Exactly-once delivery: Transport-level exactly-once is not attainable in async + crash; aim for effectively-once via idempotent ops + dedupe (request IDs) + transactional sinks.
- FLP statement: It proves no deterministic consensus with even one crash in a fully asynchronous model; timeouts assume partial synchrony.
- Perfect link abstraction: Emulated with acks/retries/seqnos, but not guaranteed under full asynchrony + crashes.

- 2PC Phase-2 behavior: After all YES and decision=COMMIT, you do not abort due to non-responses; coordinator keeps retrying until participants learn the decision.
- RAMP: Provides Read Atomic multi-partition visibility without 2PC; uses metadata + read repair; correct the “writes made visible using 2PC” line.
- Witness replicas: Attribute to Paxos/Raft groups (e.g., Spanner, CockroachDB); remove Cassandra from examples.
- CRDTs: Fix CmRDT requirements (commutative ops + causal/reliable delivery) and CvRDT lattice merge semantics.
- Linearizability vs Sequential consistency: Emphasize real-time precedence for linearizability; sequential ignores real time.
- Quorum claim (R+W>N): Clarify scope—latest completed write in absence of concurrency and with versioning; concurrent writes need conflict resolution order (e.g., LWW or timestamps).
- Raft/Multi-Paxos re-election: Leadership is stable; re-elections only on failure/suspicions; the message reduction comes from skipping prepare (Multi-Paxos) or leader pipelines.
- Failure detectors: Name Chandra–Toueg classes; mention SWIM and indirect probe as the practical baseline.
- Paxos nomenclature & Flexible Paxos: Use Prepare/Promise + Accept/Accepted; clarify intersection requirement only across phases.